

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics
Practical – III

Roll No.: T018	Name: Sachin Ramesh Mehta
Class: TYIT	Batch: 1
Date of Assignment: 14/08/2026	Date/Time of Submission: 14/08/2026

a. Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

Code & Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
✓ 7.6s

# Load Dataset

df = pd.read_csv("housing.csv")
✓ 0.1s

# Display First Five Records

print(df.head())

# Display Numerical Columns

print(df.select_dtypes(include="number").columns)
✓ 0.0s

longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23    37.88                41             880             129.0
1    -122.22    37.86                21            7099            1106.0
2    -122.24    37.85                52            1467             190.0
3    -122.25    37.85                52            1274             235.0
4    -122.25    37.85                52            1627             280.0

population  households  median_income  median_house_value
0          322         126         8.3252          452600
1         2401        1138         8.3014          358500
2          496         177         7.2574          352100
3          558         219         5.6431          341300
4          565         259         3.8462          342200
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='str')
```

```
# Calculate Correlation Matrix

correlation = df.corr(numeric_only=True)
print(correlation)
✓ 0.0s

longitude    longitude    latitude    housing_median_age    total_rooms    \
longitude    1.000000    -0.924664    -0.108197    0.044568
latitude     -0.924664    1.000000         0.011173    -0.036100
housing_median_age -0.108197    0.011173         1.000000    -0.361262
total_rooms    0.044568    -0.036100        -0.361262    1.000000
total_bedrooms  0.069608    -0.066983        -0.320451    0.930380
population     0.099773    -0.108785        -0.296244    0.857126
households     0.055310    -0.071035        -0.302916    0.918484
median_income  -0.015176    -0.079809        -0.119034    0.198050
median_house_value -0.045967    -0.144160         0.105623    0.134153

longitude    total_bedrooms    population    households    median_income    \
longitude    0.069608    0.099773    0.055310    -0.015176
latitude     -0.066983    -0.108785    -0.071035    -0.079809
housing_median_age -0.320451    -0.296244    -0.302916    -0.119034
total_rooms    0.930380    0.857126    0.918484    0.198050
total_bedrooms  1.000000    0.877747    0.979728    -0.007723
population     0.877747    1.000000    0.907222    0.004834
households     0.979728    0.907222    1.000000    0.013033
median_income  -0.007723    0.004834    0.013033    1.000000
median_house_value  0.049686    -0.024650    0.065843    0.688075

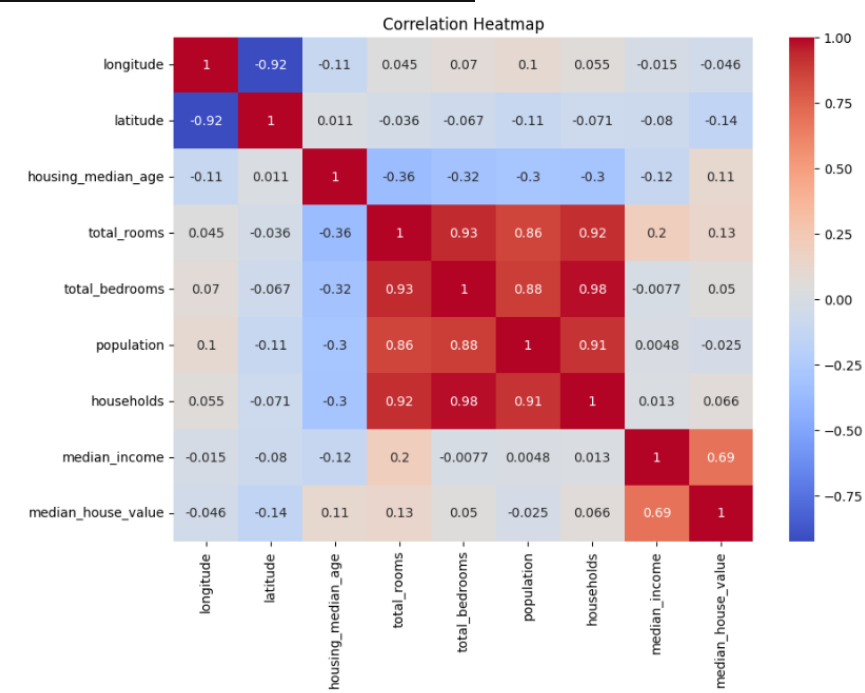
# Display Correlation Heatmap

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

plt.show()
✓ 0.9s
```



b. Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

Code & Output:

```
# Import Libraries
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

✓ 13.7s

```
# Load Dataset
```

```
df = pd.read_csv("housing.csv")
```

✓ 0.0s

```
# Select Independent and Dependent Variables
```

```
X = df[["median_income"]]
y = df[["median_house_value"]]
```

✓ 0.2s

```
# Split Dataset into Training and Testing Data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

✓ 0.1s

```
# Create Linear Regression Model
```

```
model = LinearRegression()
```

✓ 0.0s

```
# Train the Model
```

```
model.fit(X_train, y_train)
```

✓ 0.5s

▼ LinearRegression ⓘ ?

- Parameters
- Fitted attributes

```
# Display Coefficient and Intercept
```

```
print("Coefficient:", model.coef_[0])  
print("Intercept:", model.intercept_)
```

✓ 0.0s

Coefficient: 41933.84939381274
Intercept: 44459.729169078666

```
# Predict House Values
```

```
y_pred = model.predict(x_test)
```

✓ 0.0s

```
# Display First 10 Predictions
```

```
print("Predicted Values:")
```

```
print(y_pred[:10])
```

✓ 0.0s

Predicted Values:
[114958.91676996 150606.88213964 190393.71844449 285059.38345102
200663.31816103 242165.24890609 257647.22610228 199229.18051176
245893.1681172 384677.43607096]

```
# Calculate Mean Squared Error
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
```

✓ 0.0s

Mean Squared Error: 7091157771.76555

```
# Calculate R2 Score
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("R2 Score:", r2)
```

✓ 0.0s

R2 Score: 0.45885918903846656

```
# Scatter Plot with Regression Line
```

```
plt.figure(figsize=(8, 5))
```

```
sns.regplot(  
    x="median_income",  
    y="median_house_value",  
    data=df  
)
```

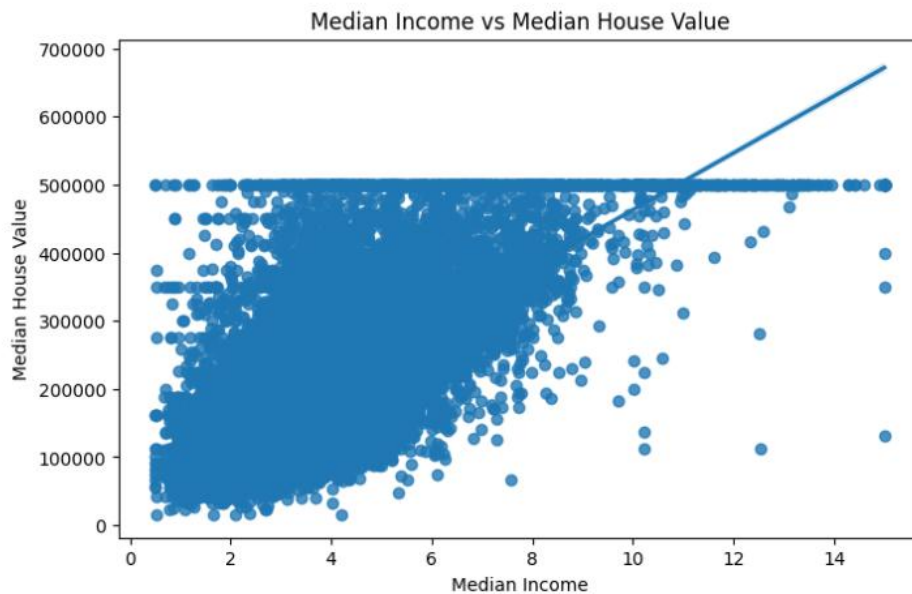
```
plt.xlabel("Median Income")
```

```
plt.ylabel("Median House Value")
```

```
plt.title("Median Income vs Median House Value")
```

```
plt.show()
```

✓ 1.9s



Extra Questions:

Q.1 Calculate the complete correlation matrix for all numerical variables.

Identify:

The strongest positive correlation.

The strongest negative correlation.

The weakest correlation.

Display the results.

Code & Output:

```
import pandas as pd

# Load dataset
df = pd.read_csv("housing.csv")

# Select numerical columns
numerical_data = df.select_dtypes(include=["int64", "float64"])

# Correlation matrix
corr_matrix = numerical_data.corr()

print("Complete Correlation Matrix:\n")
print(corr_matrix)

# Remove self-correlation (1.0)
corr_copy = corr_matrix.copy()
```

```

for i in range(len(corr_copy)):
    corr_copy.iloc[i, i] = None

# Strongest positive correlation
max_corr = corr_copy.unstack().idxmax()
max_value = corr_copy.unstack().max()

# Strongest negative correlation
min_corr = corr_copy.unstack().idxmin()
min_value = corr_copy.unstack().min()

# Weakest correlation
weak_corr = corr_copy.abs().unstack().idxmin()
weak_value = corr_copy.abs().unstack().min()

print("\nStrongest Positive Correlation:")
print(max_corr, "=", max_value)

print("\nStrongest Negative Correlation:")
print(min_corr, "=", min_value)

print("\nWeakest Correlation:")
print(weak_corr, "=", weak_value)

```

✓ 0.2s

Complete Correlation Matrix:

	longitude	latitude	housing_median_age	total_rooms	\
longitude	1.000000	-0.924664	-0.108197	0.044568	
latitude	-0.924664	1.000000	0.011173	-0.036100	
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	
total_rooms	0.044568	-0.036100	-0.361262	1.000000	
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	
population	0.099773	-0.108785	-0.296244	0.857126	
households	0.055310	-0.071035	-0.302916	0.918484	
median_income	-0.015176	-0.079809	-0.119034	0.198050	
median_house_value	-0.045967	-0.144160	0.105623	0.134153	

	total_bedrooms	population	households	median_income	\
longitude	0.069608	0.099773	0.055310	-0.015176	
latitude	-0.066983	-0.108785	-0.071035	-0.079809	
housing_median_age	-0.320451	-0.296244	-0.302916	-0.119034	
total_rooms	0.930380	0.857126	0.918484	0.198050	
total_bedrooms	1.000000	0.877747	0.979728	-0.007723	
population	0.877747	1.000000	0.907222	0.004834	
households	0.979728	0.907222	1.000000	0.013033	
median_income	-0.007723	0.004834	0.013033	1.000000	
median_house_value	0.049686	-0.024650	0.065843	0.688075	

	median_house_value
longitude	-0.045967
latitude	-0.144160
housing_median_age	0.105623
total_rooms	0.134153
total_bedrooms	0.049686
population	-0.024650
households	0.065843
median_income	0.688075
median_house_value	1.000000

```
Strongest Positive Correlation:  
('total_bedrooms', 'households') = 0.9797282708045709  
  
Strongest Negative Correlation:  
('longitude', 'latitude') = -0.9246644339150368  
  
Weakest Correlation:  
('population', 'median_income') = 0.004834345627653126
```

Q.2 Build a regression model using:

X=median_income

X=housing_median_age

X=total_rooms

X=population

X=households

Y=median_house_value

Evaluate the model using:

- R2

- Mean Squared Error

Compare this model with the simple regression model using only median_income.

Code & Output:

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error, r2_score  
  
# Load dataset  
df = pd.read_csv("housing.csv")  
  
# Multiple Regression  
X_multi = df[  
    [  
        "median_income",  
        "housing_median_age",  
        "total_rooms",  
        "population",  
        "households"  
    ]  
]  
  
y = df["median_house_value"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X_multi,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

```

model_multi = LinearRegression()

model_multi.fit(X_train, y_train)

y_pred_multi = model_multi.predict(X_test)

print("Multiple Regression")

print("R2 Score:",
      |      r2_score(y_test, y_pred_multi))

print("Mean Squared Error:",
      |      mean_squared_error(y_test, y_pred_multi))

# Simple Regression

X_simple = df[["median_income"]]

X_train, X_test, y_train, y_test = train_test_split(
    |      X_simple,
    |      y,
    |      test_size=0.2,
    |      random_state=42
    )

model_simple = LinearRegression()

model_simple.fit(X_train, y_train)

y_pred_simple = model_simple.predict(X_test)

print("\nSimple Regression")

```

```

print("R2 Score:",
      |      r2_score(y_test, y_pred_simple))

print("Mean Squared Error:",
      |      mean_squared_error(y_test, y_pred_simple))
✓ 0.2s

```

```

Multiple Regression
R2 Score: 0.5490012551168503
Mean Squared Error: 5909928044.702565

Simple Regression
R2 Score: 0.45885918903846656
Mean Squared Error: 7091157771.76555

```

Q.3 Find the correlation between:

- total_rooms
- median_house_value

Code & Output:

```
import pandas as pd

# Load dataset
df = pd.read_csv("housing.csv")

correlation = df["total_rooms"].corr(
    df["median_house_value"]
)

print(
    "Correlation between total_rooms and median_house_value:"
)

print(correlation)
```

✓ 0.1s

```
Correlation between total_rooms and median_house_value:
0.1341531138065631
```